WNDRVR
SUPPORT NETWORK

VXWORKS 7 SSH PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 SSH Programmer's Guide, SR0650

14 August 2020

1. OVERVIEW

1. Introduction to VxWorks SSH

VxWorks® SSH, an implementation of the Secure Shell protocol (SSH), provides secure remote login, file transfer, and port forwarding over a network.

> Note: Port forwarding is supported by the SSH server module when a connecting SSH client provides local port forwarding parameters.
>
> Port forwarding is not supported if the SSH server uses an SSH subsystem (acting as an SSH client).

VxWorks SSH functions as both an SSH server and an SSH File Transfer Protocol (SFTP) server.

VxWorks SSH also includes an SSH client and an SFTP client.

VxWorks SSH supports password and public key authentication for IPv4 and IPv6 connections as described in the 1.5 and 2.0 IETF draft versions of the Secure Shell (SSH) Protocol. Version 2.0 is not a superset of version 1.5, and each use their own support for various security features. For example, version 1.5 of the SSH Connection Protocol supports terminal connections and port forwarding. Version 2.0 provides this support, as well as SFTP support.

VxWorks SSH uses the cryptography services of the VxWorks Cryptography Libraries. For more information, see the VxWorks 7 Cryptography Libraries Programmer's Guide.

2. SSH Technology Overview

The SSH protocol implementation consists of several protocol elements.

SSH Transport Protocol

This protocol provides a secure channel used by other SSH sub-protocols. The SSH Transport Protocol handles secure key exchange, server authentication, encryption, replay, and integrity protection. RFC 4253 defines the SSH Transport Protocol.

SSH User Authentication Protocol

This protocol is a general purpose protocol for user authentication. It requires a secure channel provided by the SSH Transport Protocol. RFC 4252 defines the SSH User Authentication protocol.

SSH Connection Protocol

This protocol provides interactive login sessions and forwards TCP connections. It can run the SSH File Transfer Protocol as a subsystem. RFC 4254 defines the SSH Connection Protocol.

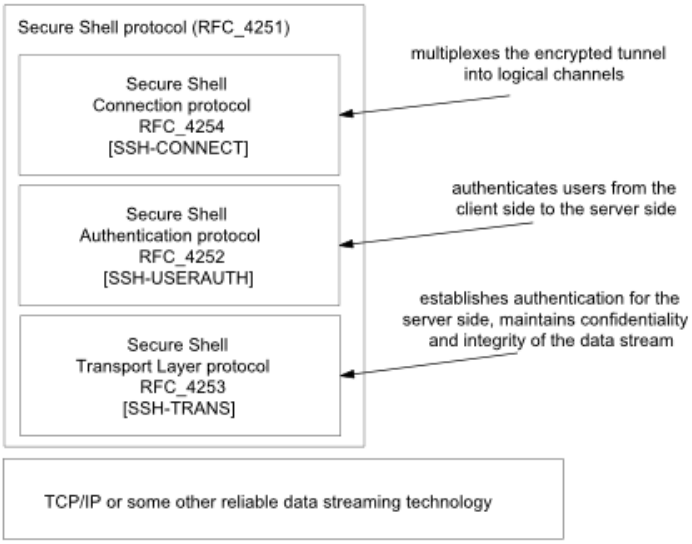
SSH File Transfer Protocol (SFTP)

This protocol provides the same services as the FTP protocol to provide secure file system access with one main difference. In most cases, SFTP runs in a secure channel as a subsystem of the SSH Connection Protocol. Standard non-secure FTP runs over clear-text TCP connections.

> Note: This protocol is not a formal part of the SSH protocol suite, but it is designed to work in a secure environment such as that provided by SSH.

The SSH protocol elements are as follows:

WNDRVR



User Authentication Module Features on page 2 The SSH Authentication Protocol supports password, public key, and host-based authentication.
Connection Module Features on page 2 VxWorks SSH is pre-integrated with the VxWorks shell.
SSH File Transfer Protocol Module Features on page 3 VxWorks SSH supports both the SSH File Transfer Protocol (SFTP) client and server.
SSH Cryptographic Support on page 3 VxWorks SSH supports different cryptographic technologies for SSH version 2.0 and SSH version 1.5.

2.1. User Authentication Module Features

The SSH Authentication Protocol supports password, public key, and host-based authentication.

Separate modules handle the validation of user IDs, passwords, and public keys. For more information see, Authentication on page 18.

Parent topic: SSH Technology Overview on page 1

2.2. Connection Module Features

VxWorks SSH is pre-integrated with the VxWorks shell.

For more information about shells, integrating a proprietary shell with VxWorks SSH, or using multiple shells, see Integration with Shells on page 23. The connection module supports the following SSH features as implemented by the protocol:

- terminal connections
- remote execution of commands using SSH exec
- port forwarding
- SSH subsystems

Note:

Port forwarding is supported by the SSH server module when a connecting SSH client provides local port forwarding parameters.

Port forwarding is not supported if the SSH server uses an SSH subsystem (acting as an SSH client).

The **exec** feature is invoked from the SSH client. Port forwarding is also set up from the SSH client. For details on using these features, see your SSH client documentation. For information on implementing an SSH subsystem, see Server Side SSH Subsystem on page 26 and Client Side SSH Subsystem on page 27.

Parent topic: SSH Technology Overview on page 1

2.3. SSH File Transfer Protocol Module Features

VxWorks SSH supports both the SSH File Transfer Protocol (SFTP) client and server.

SFTP provides basic file management over secure SSH connections. With this module, SSH supports:

- file transfers in both directions
- directory listings
- creation, removal, and management of directories and files

If the underlying file system does not support access control, you can implement a callback to handle access control on file operations. For more information, see File Systems and SFTP on page 28.

Parent topic: SSH Technology Overview on page 1

2.4. SSH Cryptographic Support

VxWorks SSH supports different cryptographic technologies for SSH version 2.0 and SSH version 1.5.

VxWorks SSH supports the following Symmetric Cipher Algorithm options for SSH version 2.0.

Symmetric Cipher Algorithm Type	Standards
AES128-CBC	NIST FIPS 197
AES192-CBC	NIST FIPS 197
AES256-CBC	NIST FIPS 197
AES128-CTR	RFC 3344
AES192-CTR	RFC 4344
AES256-CTR	RFC 4344
AES128-GCM	As implemented by OpenSSH
AES256-GCM	As implemented by OpenSSH
DES-56-CBC	NIST FIPS 46-3
3DES-168-CBC	NIST FIPS 46-3
Arcfour (ARC4 - 128 bit key)	RFC 4345
Arcfour-128	RFC 4345

Symmetric Cipher Algorithm Type	Standards
(128 bit key, but skips 1536 first bytes of the key stream)	
Blowfish-128	
CAST-128	RFC 2144
None (no encryption performed, data sent in clear text)	

VxWorks SSH supports the following Hash and Message Digest options and Message Authentication Code options for SSH version 2.0.

Hash and Message Digest Type	Standards
HMAC-MD5-96	RFC 1321
HMAC-SHA1	
HMAC-SHA1-96	
HMAC-SHA2-256	RFC 6688
HMAC-SHA2-512	RFC 6688
None (no encryption performed, data sent in clear text)	

VxWorks SSH supports the following Public Key Encryption and Key Exchange options for SSH version 2.0.

Public Key Encryption and Key Exchange Type	Standards
Diffie-Hellman key exchange, including: • diffie-hellman-group1-sha1 (768 bits) • diffie-hellman-group14-sha1 (2048 bits) • diffie-hellman-group-exchange-sha1 • diffie-hellman-group-exchange-sha256	RFC 2631
RSA key exchange, including: • rsa1024-sha1 (1024 bits) • rsa2048-sha256 (2048 bits)	RFC 4432
ECDH key exchange, including: • ecdh-sha2-nistp256 • ecdh-sha2-nistp384	RFC 4432

Public Key Encryption and Key Exchange Type	Standards
• ecdh-sha2-nistp521	

VxWorks SSH supports the following Symmetric Cipher Algorithm options for SSH version 1.5.

Symmetric Cipher Algorithm Type	Standards
DES-56-CBC	NIST FIPS 46-3
3DES-112-CBC	NIST FIPS 46-3
Arcfour (ARC4)	RFC-4345
Blowfish-cbc	
None	

Parent topic: SSH Technology Overview on page 1

3. RFCs

Wind River SSH complies with a number of IETF RFCs.

The following supported RFCs are accessible from the IETF website.

RFC	Description
RFC 4250	*The Secure Shell (SSH) Protocol Assigned Numbers*
RFC 4251	*The Secure Shell (SSH) Protocol Architecture*
RFC 4252	*The Secure Shell (SSH) Authentication Protocol*
RFC 4253	*The Secure Shell (SSH) Transport Layer Protocol* Excluding 6.2 Compression
RFC 4254	*The Secure Shell (SSH) Connection Protocol* Excluding: • 6.3 X11 Forwarding • 6.4 Environment Variable Passing • 6.5 Starting a Shell or a Command, (no “exec message” at the end) • 6.8 Local Flow Control • 6.9 Signals

RFC	Description
	• 6.10 Returning Exit Status
RFC 4419	*Diffie-Hellman Group Exchange for the Secure Shell (SSH) Transport Layer Protocol*
RFC 4432	*RSA Key Exchange for the Secure Shell (SSH) Transport Layer Protocol*
RFC 4716	*The Secure Shell (SSH) Public Key File Format*
N/A	*The Secure Shell (SSH) File Transfer Protocol, IETF Draft (draft-ietf-secsh-filexfer)* Only SFTP version 3

2. SSH SERVER

1. Implementing an SSH Server

Implementation Workflow

The basic steps required for implementing an SSH server are described in the following topics:

1. Configure VSB for SSH and User Management on page 7
2. Create a VIP and Configure SSH on page 7
3. Configure VIP for Key Store and Key Commands on page 9
4. Configure VIP for User Database on page 9
5. Generate RSA and DSA Keys and Add to Key Store on page 10
6. Add Users, Add Login Security, and Test SSH Connection on page 12

2. Configure VSB for SSH and User Management

Configure your VxWorks source build (VSB) project with support for the SSH server and client, and user management.

Procedure

1. Create a VSB project.

   Consider using the VxWorks simulator (VxSim) with the host file system for your initial work with SSH. The examples provided throughout this workflow reflect this usage, but are useful for hardware targets as well.

2. Enable the **IPNET_SSH** option.
3. For user management and login support, enable **USER_MANAGEMENT**.
4. Build the VSB.

Next Steps

Create a VIP and Configure SSH on page 7

3. Create a VIP and Configure SSH

Before You Begin

Configure VSB for SSH and User Management on page 7

Procedure

1. Create a VxWorks image project (VIP) based on your SSH VSB project.

   You will need the kernel shell, so either use **PROFILE_DEVELOPMENT** when you create your VIP, or add **INCLUDE_SHELL** after you have created it.

2. Add **INCLUDE_SSH**.

   When you do so, you get a configuration error. This is because the default KEP backend is **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR**, and the value for the **SEC_VAULT_KEY_ENCRYPTING_PW** parameter is not set.

   For more information about the KEP backend, including alternatives to the default, see the VxWorks 7 Cryptography Libraries Programmer's Guide.

3. Set **SEC_VAULT_KEY_ENCRYPTING_PW**.

   This configuration parameter defines the password used to encrypt keys in the key store. The parameter value should be a random string of at least 40 characters containing both lower and upper case letters, numbers, and special characters. No default is provided by intention, to ensure that unique values are used. You must define a password, otherwise your VIP will not build.

4. For SSH client support, add **INCLUDE_IPSSH_CLIENT_CMD**.
5. Set the **SSH_SFTP_START_DIR** parameter to a working directory for SSH and SFTP client connections.

   By default, it is set to "/ram". See Configuring Root and Initial Working Directories for the Client on page 13.

6. Set the **SSH_DSA_KEY_ID** and **SSH_RSA_KEY_ID** configuration parameters.

   CAUTION: **When you add keys to the VxWorks key store after booting your system, you must use these values for the key IDs. If the values do not match, the SSH server will not be able to fetch the keys using the key IDs, and the server will not start. See** Generate RSA and DSA Keys and Add to Key Store on page 10**.**

7. Set **SSH_RSA_KEYEX_KEY_ID** to the key ID that will be used in the key store for the key used in the RSA key exchanges.

   At runtime, this key ID is used to fetch the RSA key from the Secure Key DB used in RSA key exchange. You can use the same key ID as **SSH_RSA_KEY_ID** if the same key will be used as for the server authentication, or a different key.

8. Set other **INCLUDE_SSH** parameters, as required for your system.

   There are numerous SSH configuration parameters. See the **INCLUDE_SSH** entry in the API and Build Configuration Reference.

   CAUTION: **The default version of SSH is v2. Wind River recommends that you do not use SSH v1, as it is not considered secure. In addition, SSH v1 is incompatible with FIPS 140-2, and they should not be used together.**

Next Steps

Configure VIP for Key Store and Key Commands on page 9

If your system is unable to accommodate the the maximum number of connections as defined in with **SSH_MAX_CLIENTS**, you may need to increase the value assigned to the **NUM_FILES** parameter (of **INCLUDE_IO_BASIC**).

4. Configure VIP for Key Store and Key Commands

Before You Begin

Create a VIP and Configure SSH on page 7

Procedure

1. In your VIP project, set **SEC_VAULT_FILE_ROOT** to the directory below which the key store will be created.

   With a VxSim target you can use the host file system, for example by setting the parameter to "host:C:/tmp". For a hardware target, set it to a local storage file system directory.

   At runtime, the subdirectories **keys** and **vault** are created automatically under this directory when keys or passwords are imported (see Generate RSA and DSA Keys and Add to Key Store on page 10).

2. Add the components that provide the key generation and key store utility shell commands.
   - **INCLUDE_SEC_KEY_STORE_CMD** for the keyStore management commands.
   - **INCLUDE_GENRSA_CMD** for the OpenSSL genrsa command.
   - **INCLUDE_GENDSA_CMD** for the OpenSSL gendsa command.
   - **INCLUDE_DSAPARAM_CMD** for the OpenSSL dsaparam command.

Next Steps

Configure VIP for User Database on page 9

5. Configure VIP for User Database

About This Task

For detailed information about user management and the user database, see the *VxWorks 7 Security Programmer's Guide: User Authentication and Management*.

Before You Begin

Configure VIP for Key Store and Key Commands on page 9

Procedure

1. Add the **INCLUDE_USER_IDENTIFICATION** component.

   This component provides support for creating and deleting user accounts, for updating passwords, and so on.

2. Optionally, change the configuration parameters **DEFAULT_PRIMARY_GROUP_ID** and **VXWORKS_LOGIN_PROMPT**.
3. Add **INCLUDE_USER_DATABASE**.
4. Set **UDB_STORAGE_PATH** to a location and file name for the user database.

WNDRVR

For example, with VxSim and the host file system, you could use **"host:C:/tmp/myUserDb.txt"**.

5. Set the secret hash key that will be used for computing the HMAC with the **UDB_HASH_KEY** parameter.

   The hash key should have a minimum of 256 random bits (40 ASCII characters), encoded in *\xnn* format, where *nn* can vary from 00 to ff. The longer the hash key, the higher the quality of cryptographic hashing. For example, the following could be assigned to **UDB_HASH_KEY** (without line breaks):

```
"\x48\x61\x6d\x6c\x65\x74\x2e\x20\x54\x6f\x20\x62\x65\x2c\x20\x6f\x72\x20\x6e\x6f\x74\x20\x74\x
6f\x20\x62\x65\x2d\x20\x74\x68\x61\x74\x20\x69\x73\x20\x74\x68\x65
```

   CAUTION: **For security reasons, do not use the hash key provided in this example. It is for illustrative purposes only. Using this hash key may compromise the security of your system.**

6. Set the **UDB_HASH_KEY_LEN** parameter, or use the default value of 256 (the minimum value).

   CAUTION: **If the INCLUDE_SECURITY component is included in a VIP, but the UDB_STORAGE_PATH or UDB_HASH_KEY parameters are left set to null, at runtime the user database manager will not be initialized properly and the user database will not be created.**

7. Add components for file system storage, and code for file system initialization at boot time.

   For initial system testing with the VxWorks simulator (VxSim), consider using the host file system (as illustrated with the preceding and following examples).

   For information about file system support, see the *VxWorks 7 File Systems and I/O Programmer's Guide: I/O Devices, Named Files, and File Systems*.

8. Add **INCLUDE_DISK_UTIL** for the standard set of file system utilities (cd, copy, ll, mkdir, and so on).
9. Build your VIP.

Next Steps

Generate RSA and DSA Keys and Add to Key Store on page 10

6. Generate RSA and DSA Keys and Add to Key Store

About This Task

For detailed information about the key store see the VxWorks 7 Cryptography Libraries Programmer's Guide.

For information about the genrsa, dsaparam, gendsa, and keyStore commands, use any command with the -help option from the kernel shell (with the command interpreter). For more detailed information, see the genrsa, dsaparam, and gendsa command reference entries in the documentation on https://www.openssl.org.

CAUTION: **When you add the DSA and RSA keys to the key store, you must use the key IDs that you assigned to the SSH_DSA_KEY_ID and SSH_RSA_KEY_ID configuration parameters (in Configure VIP for Key Store and Key Commands on page 9). If the values do not match, the SSH server will not be able to fetch the keys through the key IDs, and it will not start.**

WNDRVR

Before You Begin

Configure VIP for User Database on page 9

Procedure

1. Boot your target system with VxWorks.
2. Change to the command interpreter and use genrsa to create the RSA key file.

   For example (using the -rand option with a random data file):

```
-> cmd
[vxWorks *]#
```

```
[vxWorks *]# genrsa -out default_rsa.pem -rand host:C:/tmp/seed.txt 2048
0 semi-random bytes loaded
Generating RSA private key, 2048 bit long modulus
e is 65537 (0x10001)
[vxWorks *]#
```

   The file **default_rsa.pem** is created in the current directory.

3. Use dsaparam to generate the DSA parameters.

   For example:

```
[vxWorks *]# dsaparam -out dsaparam.pem -rand host:C:/tmp/seed.txt 2048
0 semi-random bytes loaded
Generating DSA parameters, 2048 bit long prime
This could take some time
....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*
...........+...+.........................+......................+...+..+......+..........+...
..+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*
[vxWorks *]#
```

4. Use gendsa to generate a DSA key.

   For example:

```
[vxWorks *]# gendsa -out default_dsa.pem -rand host:C:/tmp/seed.txt host:dsaparam.pem
0 semi-random bytes loaded
Generating DSA key, 2048 bits
[vxWorks *]#
```

5. Use the keystore import command to add the keys to the key store.

   For example:

```
[vxWorks *]# keyStore import default_rsa default_rsa.pem
[vxWorks *]# keyStore import default_dsa default_dsa.pem
```

   As part of the import process, sub-directories of keys are created using the key ID names.

```
[vxWorks *]# ls host:C:/tmp/keys
host:C:/tmp/keys/.
host:C:/tmp/keys/..
host:C:/tmp/keys/default_dsa
host:C:/tmp/keys/default_rsa
```

Next Steps

Add Users, Add Login Security, and Test SSH Connection on page 12

7. Add Users, Add Login Security, and Test SSH Connection

Before You Begin

Generate RSA and DSA Keys and Add to Key Store on page 10

Procedure

1. At the kernel shell prompt, change to the C interpreter, and use udbReset( ) to create the user database.

   For example:

   ```
   [vxWorks *]# C
   -> udbReset
   ```

   The udbInfoShow( ) command can be used to display information about the database.

2. Add users with userAdd( ).

   For example:

   ```
   ->  userAdd "poohbear", "honey"
   ```

3. Reconfigure your VIP project to include user login security with **INCLUDE_SHELL_SECURITY**, and then rebuild your project.
4. Reboot your target with the new VxWorks image.
5. Test your configuration by logging on to the SSH server system from a client system.

   In this example, an SSH connection is made from the VxSim target 192.168.200.2 to the VxSim target 192.168.200.1, and then the connection is closed.

   ```
   -> cmd
   [vxWorks *]# ssh poohbear@192.168.200.1

   poohbear's password:


   -> pwd
   ```

```
host:C:/WindRiver/workspace/VXSIM-SSH-VIP/default
value = 68 = 0x44 = 'D'
-> exit

Au revoir!
[vxWorks *]# pwd
host:C:/WindRiver/workspace/vxsim-ssh-client-VIP/default
[vxWorks *]#
```

8. Add Users Automatically at Boot Time

Procedure

1. Add code to your VIP project automatically create user accounts at boot time.

   For example, open the **usrAppInit.c** file located in your project directory, and add these lines after **#include <vxWorks.h>**:

   ```
   #include "udbMgr.h"
   #include "userIdentLib.h"
   ```

   Then add the following code to the **usrAppInit( )** function (after your file system initialization code):

   ```
   udbReset();
   userAdd("ssh","secret",0,0);
   ```

2. Rebuild your VIP project.

9. Configuring Root and Initial Working Directories for the Client

The session root directory is the top directory the sftp client sees in the file tree. Use the SSH_SFTP_START_DIR parameter to set the current working directory that clients have available when they connect.

About This Task

The sftp server presents the session root directory as **/** to the sftp client. If the start directory is a sub-directory of the session root directory, the sftp server presents that directory as the start directory, otherwise the sftp server presents **/** as the start directory.

For example, given these settings, the Secure FTP server supports and determines the following combinations of start directory and root directory:

Configured start and determined session root directories	Start and root directories as seen by an Secure FTP client
Start: **/ram**	Start: **/ram**

Configured start and determined session root directories	Start and root directories as seen by an Secure FTP client
Root: not set	Root: **/**
Start: **/ram/test** Root: /ram	Start: **/test** Root: **/**
Start: **/ram** Root: **/a/b/c**	Start: / Root: **/**

10. Configuring SFTP Session and Root Trespass Protection

Wind River SSH supports Secure FTP server session root and trespass protection mechanisms in the same manner as the FTP server does.

About This Task

The SFTP server allows you to:

- Make the SFTP session root configurable by using the authentication callback **ipssh_auth_complete_cb( )** defined in: *installDir* **/vxworks-7/pkgs_v2/net/ipnet-***version* **/ssh-***x.x.x.x* **/cfg/ipssh_config.c**
- Set the initial working directory at client session startup with the use of the **SSH_SFTP_START_DIR** configuration parameter.
- Implement root trespass checking for file operations in SFTP with the use the **ipcom_shell_ctx_add( )** function that adds a IPSSH_SFTP_ROOT_DIR parameter to the shell context.

3. SSH CLIENT

1. SSH Client for VxWorks

VxWorks supports the SSH client version 2.

Version 1 is not supported. The client is comparable to OpenSSH in functionality. It works with the following servers:

- VxWorks 6*x* and VxWorks 7 SSH servers
- Linux SSH server
- Windows SSH server

The following features are not supported with this release:

- authentication agent connection forwarding
- dynamic port forwarding

For information about the VxWorks SSH server, see the *VxWorks 7 SSH Programmer's Guide on page 1*.

2. Configuring VxWorks With an SSH Client

Configure VxWorks with SSH client support in both your VSB and VIP projects.

Procedure

1. Create a VSB project.
2. Add the **SSH_CLIENT** layer.
3. Build the VSB project.
4. Create a VIP project based on the VSB project.
5. Add the **INCLUDE_IPSSH_CLIENT_CMD** component.

   An SSH VIP project supports both the client and server. You will therefore get a configuration error because the key encrypting password provider **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR** is included by default for the SSH server, and the **SEC_VAULT_KEY_ENCRYPTING_PW** configuration parameter must be set.

6. Set the **SEC_VAULT_KEY_ENCRYPTING_PW** configuration parameter, or select and configure a different key encrypting password provider.

   If you are using this project just for an SSH client, simply set the parameter and proceed. If you are using the project for both an SSH client and server, you can use the default **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR**key encrypting password provider, or a different one (see the topic Key-Encrypting Password Providers in the *VxWorks 7 Security Programmer's Guide: Password and Login Policies*).

7. Build the VIP project.

3. Using the SSH Client

Use the SSH client in the standard manner to connect with an SSH server.

Procedure

- For SSH client command syntax, use the ssh command (with or without the -help option) from the kernel shell (using the command interpreter).

```
-> cmd
[vxWorks *]# ssh
```

- Use the ssh command with or without options to establish a secure connection with an SSH server.

  For example:

```
[vxWorks *]# ssh xeno@192.168.200.1
```

4. SSH APPLICATIONS

1. Developing SSH Kernel Applications

VxWorks SSH applications must be created as kernel applications that execute in kernel-memory space in kernel mode. They cannot be created as a real-time process (RTP).

The VxWorks SSH server runs as a daemon process that starts at boot time. Generally, modifying the way the daemon operates requires changing configuration options and modifying initialization and callback routines. VxWorks SSH also includes an SFTP client with its own API, see SSH Libraries on page 17. The SFTP client can also be run in the kernel shell using the command interpreter, see Command-Line Usage on page 29.

VxWorks applications that execute in the kernel are created as relocatable object modules. When you build a kernel-based application module, user code is linked to the required VxWorks libraries, and an ELF binary is produced. Kernel-based application modules use VxWorks facilities by including header files that define VxWorks interfaces and data structures.

Kernel-based application modules can be either of the following:

- Downloaded and dynamically linked to VxWorks by the object module loader.
- Statically linked to VxWorks, making them part of the system image.

VxWorks SSH includes a server for SSH and SFTP and clients for SSH and SFTP. Modifications to server functionality can be done using a combination of the following techniques:

- Change kernel configuration parameters. See Add Users Automatically at Boot Time on page 13.
- Add initialization and configuration code to **ipssh_configure( )**. See SSH Initialization on page 18.
- Change callback routines that implement specific functions. For example, user authentication is handled by callback routines for public key authentication and password authentication. See Authentication on page 18.
- Dynamically change the configuration at runtime. See Run-time Configuration on page 23.

The SFTP client API can be used to connect to an external SFTP server. See Creating an SFTP Client Connection on page 27.

Note: VxWorks SSH cannot run as a real-time process (RTP).

2. SSH Libraries

The SSH libraries include connection, configuration, and SFTP functions.

ipssh

Provides connection, fingerprint generation, task, and session management functions.

ipssh_config

Provides configuration, user, and key management functions.

ipssh_sftp_clt

Provides SFTP connection and session management functions.

For detailed information, refer to the VxWorks 7 Kernel Network API Reference.

3. SSH Initialization

The VxWorks SSH daemon starts at boot time. **ipssh_config.c** contains the **ipssh_configure( )** function, which performs initialization before the daemon starts. Modify this function to change initialization options.

About This Task

Callback functions that can be used to change run-time operation are also located in **ipssh_config.c**. Modify the functions as required. The callback functions are described in the ipssh_config( ) API reference entries in the the Kernel Network API Reference page. The names of the callback functions all end with **_cb**; for example, **ipssh_validate_pubkey_cb( )**.

4. Authentication

The SSH protocol uses different types of authentication at different stages of the client-server negotiation.

As part of the transport protocol, the client authenticates the server by means of a public key. Most clients will store a list of trusted servers and their keys, and provide a means for adding new servers to that list.

Once the transport has been set up, the server authenticates the client to determine whether it will grant access, and what privileges are available. This is normally done through a user name and password combination or through a public key presented by the client. For public key authentication, the server must have a way of storing trusted client user names and their associated keys.

Server Authentication on page 18 The SSH protocol requires the server to have a server key pair for authentication during transport negotiation. VxWorks SSH uses either a DSA or RSA key pair.
RSA Key Size on page 19 There are two keys involved in SSHv1 connections, the host key and the server key.
User Authentication on page 19 The SSH protocol allows different methods to be used for user authentication. The supported methods are authentication using a password or a public key pair.
Configuring Client Authentication Using a Public Key Pair on page 20 You can generate your own public keys.
Configuring Client Authentication Using Host-Based Mode on page 22 You can generate your own public keys.

4.1. Server Authentication

The SSH protocol requires the server to have a server key pair for authentication during transport negotiation. VxWorks SSH uses either a DSA or RSA key pair.

The kernel configuration parameters **SSH_DSA_KEY_ID** and **SSH_RSA_KEY_ID** set the key ID used to fetch the DSA and RSA server keys, respectively.

Use any suitable method to generate and store the private keys. Depending on the needs of the application, this could mean generating the keys externally and loading them onto the target, or it could mean generating the keys directly on the target using cryptography routines available as part of the VxWorks Cryptography Libraries.

By default, **SSH_DSA_KEY_ID** and **SSH_RSA_KEY_ID** parameters are set by the default key IDs of **default_dsa** and **default_rsa**, respectively.

CAUTION: **The values assigned to SSH_DSA_KEY_ID and SSH_RSA_KEY_ID must match the keyId values that you created when you added they keys to the key store. If the values do not match, the SSH server will not be able to fetch the keys through the key IDs, and will not start.**

Parent topic: Authentication on page 18

4.2. RSA Key Size

There are two keys involved in SSHv1 connections, the host key and the server key.

VxWorks SSH uses the key that you configure as the host key. VxWorks SSH automatically generates a corresponding server key. In order to be compliant with RFC 4253, the minimum size of the RSA server key is 512 bits. The host key must be larger than the server key, so you must configure the host key to be at least 768 bits in size. We recommend a 1024 bit key. It is not possible to use a 512 bit key as an SSHv1 host key.

VxWorks SSH uses the same RSA key for both v1 and v2 connections.

Use the **ipssh_load_rsa_key( )** function to load the RSA server key.

Use the genrsa command to generate the host key.

Parent topic: Authentication on page 18

4.3. User Authentication

The SSH protocol allows different methods to be used for user authentication. The supported methods are authentication using a password or a public key pair.

To set the types of user authentication that will be used, and in what order, set the following kernel configuration parameters:

- SSH_AUTH_PW_ALLOWED
- SSH_AUTH_PW_REQUIRED
- SSH_AUTH_PUB_KEY_ALLOWED
- SSH_AUTH_PUB_KEY_REQUIRED
- SSH_AUTH_PUB_KEY_FIRST

Note: For user and password authentication, SSH uses the VxWorks user management facility. More information about VxWorks user authentication and management can be found in the *VxWorks 7 Application Programmer's Guide:User Authentication and Management*.

Public Key Authentication on page 19 For public key authentication, use the callback function **ipssh_validate_pubkey_cb( )** to validate a public key sent by an SSH client.
Host-based Authentication on page 20

Parent topic: Authentication on page 18

Public Key Authentication

For public key authentication, use the callback function **ipssh_validate_pubkey_cb( )** to validate a public key sent by an SSH client.

The default behavior checks for the existence of a file named **user.pk2** in the **IPCOM_FILE_ROOT** directory (**/ram/**), where **user** is the user name to validate. This public key is used to validate a signed data stream created using the client's private key.

Note that the server must have copies of the public keys for clients before public key authentication will be successful. If the server does not have a matching key it notifies the client, which can then try password authentication.

To change the method of authentication, modify **ipssh_validate_pubkey_cb( )**. For example, you could change how public keys are stored and retrieved.

WNDRVR

Parent topic: User Authentication on page 19

Host-based Authentication

Some sites wish to allow authentication based on the host that the user is coming from and the user name on the remote host. The client requests this form of authentication by sending the message (defined in RFC 4252, section 9: *Host-Based Authentication: hostbased*). The server must verify that the host key belongs to the client host named in the message, that the given user on that host is allowed to log in, and that the signature value is a valid signature on the appropriate value by the given host key.

The client sends a host-based authentication message (**SSH_MSG_USERAUTH_REQUEST**) to the server for validation. If the server allows or requires host-based authentication (configured using the **SSH_AUTH_HOSTBASED_ALLOWED** and **SSH_AUTH_HOSTBASED_REQUIRED** configuration parameters) and the client has not yet tried host-based authentication, the client configures a host-based key (using **ipssh_sftp_clt_set_hostb_key( )**) and the server executes the host-based validation callback (configured using the **IPSSH_VALIDATE_HOSTBASED_CB** parameter).

If no callback is configured, **ipssh_validate_hostbased_example_cb( )** is used by default. **ipssh_validate_hostbased_example_cb( )** first checks to see the user on the host (username in **SSH_MSG_USERAUTH_REQUEST**) is allowed to log in based on the **hosts.conf** file in the **IPCOM_FILE_ROOT**/*host_based* directory.

> Note: You must create the folder host_based in the **IPCOM_FILE_ROOT** directory. Create and define the **hosts.conf** file before the client starts the SSH connection.

Each line of this file defines what type of user and host can log in. The file must use the following format:

hostname1 user1,user2,user3

Users *user1* , *user2* , and *user3* are allowed for the host named *hostname1* .

hostname2 +

All users are allowed for the host named *hostname2* .

+ userN

The user named *userN* is allowed for all hosts.

+ +

All users are allowed for all hosts.

To validate the host-based key, **ipssh_validate_hostbased_example_cb( )** checks the public key files under the **IPCOM_FILE_ROOT**/*host_based* directory. The naming convention for the public key files must be in the form *FQDNhostname.publicKeyFormat* . For example, *hostname* **.pk2** or *hostname* **.pem**.

Parent topic: User Authentication on page 19

4.4. Configuring Client Authentication Using a Public Key Pair

You can generate your own public keys.

About This Task

Procedure

1. Create a VSB.

   Enable the **IPNET_SSH** and **USER_MANAGEMENT** options.

2. Create a VIP.

   Include the **INCLUDE_SSH** and **INCLUDE_SECURITY** components.

3. Generate a new DSA key.

   For the OpenSSH client tools, use the following commands:

   ```
   ssh-keygen -b 1024 -t dsa -f dsa_key
   ssh-keygen -e -f dsa_key > secuser.pk2
   ```

   For the PuTTY client tools, inside PuTTYgen, click the **Generate** button. Then set the following:

   - In the **Parameters** box, for the **Type of key to generate:**, select **DSA**.
   - In the **Parameters** box, enter 2048 as the **Number of Bits in a generated key:** field.
   - In the **Key** box, enter the same passphrase into both the **Key passphrase:** and **Confirm passphrase:** boxes.
   - In the **Actions** box, click the **Save public key** button and the **Save private key** button.

4. Place the key in the correct locations.

   Place the newly generated key under **host:/d/wr/keys/dsa_key**.

   If you use PuTTY, open PuTTYgen for converting the key for usage by the PuTTY client.

   1. In the **Conversions** menu, chose import and select the dsa_key.
   2. On the **Parameters** tab, chose DSA and set the number of bits to 1024.
   3. Save the private and public keys. For the public file, set the file name to **secuser.pk2**.

   You must also copy this file to the target in the folder **/ram**.

   To accomplish this, include the following code line in **usrAppInit.c**:

   ```
   copy("host:/d/wr/keys/secuser.pk2", "/ram/secuser.pk2");
   ```

5. Connect to the host using an SSH client.

   Connect to the host with an SSH client using login **secuser**, while pointing to the appropriate private key.

   For the OpenSSH client tools, use the following command:

   ```
   ssh -i dsa_key secuser@VxWorks_target
   ```

   If you use PuTTY, point the Connection configuration for **SSH > Auth > Authentication parameters > Private key file** to the private key generated with PuTTYgen (**dsa_key.ppk**).

Parent topic: Authentication on page 18

4.5. Configuring Client Authentication Using Host-Based Mode

You can generate your own public keys.

About This Task

Procedure

1. Create a VSB.

   Enable the **IPNET_SSH** and **USER_MANAGEMENT** options.

2. Create a VIP.

   Include the following components in the VIP:

   - **INCLUDE_SSH**
   - **INCLUDE_SECURITY**
   - **INCLUDE_IPDNSC**

3. Create the folder used for host-based authentication.

   You must create a folder named **host_based** under the **IPCOM_FILE_ROOT** directory (**/ram/** by default).

   This folder will include the client public key file and configuration file (see steps 5 on page 22 and 6 on page 22).

4. Generate a DSA key to work with the version of the SSH protocol you are using (either 1.5 or 2.0).
5. Place the key in the correct location.

   For the default DSA key, you can use PuTTY as the key converter. Open PuTTYgen for converting the key for usage by the PuTTY client:

   1. In the **Conversions** menu, choose import and select the dsa_key.
   2. On the **Parameters** tab, choose DSA and set the number of bits to 1024.
   3. Save the public file.

   For the OpenSSH client tools, use the following command to convert the public key format for host-based authentication:

   ```
   ssh-keygen -e -f ssh_host_dsa_key.pub > secuser.pk2
   ```

   Place the public key under **host:/d/wr/keys/dsa_key** in the file named **secuser.pk2**.

   Copy this file to the target in the folder **IPCOM_FILE_ROOT/host_based/** (**/ram/host_based/**) with file name *fqdnClientHostname***.pk2**.

6. Create the **host/user** configuration file and copy it to the correct location.

   Create the **hosts.conf** file under **IPCOM_FILE_ROOT/host_based/** (**/ram/host_based/**).

   This file defines which host/user pairs are allowed to login (see Host-based Authentication on page 20). For example, if the client hostname is **client.domain.com** and the user is **ssh**, the **hosts.conf** file should include the following line:

WNDRVR

```
client.domain.com ssh
```

Save the file and put the **hosts.conf** file under **host:/d/wr/keys/dsa_key**.

After completing steps 3 on page 22, 5 on page 22 and 6 on page 22, you can include the following statements in **usrAppInit.c**:

```
mkdir("/ram/host_based");
copy("host:/d/wr/keys/secuser.pk2","/ram/host_based/client.domain.com.pk2");
copy("host:/d/wr/keys/hosts.conf", "/ram/host_based/hosts.conf");
```

7. Connect to the host using an SSH client.

   Connect to the host with an SSH client using the user name ssh, while pointing to the appropriate private key.

   For the VxWorks SFTP client using the default DSA key, use the following command:

```
sftp -f dsa_key.pem -u ssh VxWorks_target
```

   For the OpenSSH client tools, use the following command:

```
ssh -l ssh VxWorks_target
```

   For more information on how to support host-based authentication for the OpenSSH client, see the reference page for **openssh**.

Parent topic: Authentication on page 18

5. Run-time Configuration

The configuration parameters for the **INCLUDE_SSH** component have corresponding system variables (sysvars) that can be used to modify the run-time configuration of VxWorks SSH. For example, password authentication can be enabled or disabled dynamically by changing the value of the ipssh.auth.pw.allowed sysvar. For a list of SSH sysvars, access the *installDir* **/vxworks-7/pkgs_v2/net/ipnet-***version* **/ssh-***version* **/cfg/ipssh_config.c** file.

> Note: Changes to sysvars only takes effect after the **ipssh** daemon has been reconfigured, using either the ipd shell command or the **ipd** API. For more information on **ipd** and other IPCOM shell commands, see the VxWorks 7 Configuration and Build Guide.

6. Integration with Shells

By default, VxWorks SSH is completely integrated with the VxWorks shell.

A user that has been successfully authenticated has a terminal connection to the VxWorks shell, in the same way as when logging in using telnet. All registered shell commands are available in an SSH terminal connection.

Custom Shells on page 24 To use a custom shell instead of the default, you need to register a shell start routine using **ipcom_set_start_shell_ptr( )**.
Shell Integration Using the Socket Model on page 24 You can integrate a shell with VxWorks SSH using a socket connection between SSH and the shell.

Multiple Shells and Ports on page 26 Wind River SSH supports multiple shells or the ability to listen to multiple ports.

6.1. Custom Shells

To use a custom shell instead of the default, you need to register a shell start routine using **ipcom_set_start_shell_ptr( )**.

A typical shell workflow occurs as follows:

1. When a client logs in and requests a shell, SSH calls the shell start routine registered with **ipcom_set_start_shell_ptr( )** to start a shell.
2. During the session, SSH decrypts data from the client and sends it in clear text to the shell's standard input.
3. The shell can also send data from its standard output to SSH, which encrypts it and sends it to the SSH client.
4. If the client disconnects, SSH closes the TCP connection to the shell. If the shell terminates, it closes the TCP connection to the SSH server.

Parent topic: Integration with Shells on page 23

6.2. Shell Integration Using the Socket Model

You can integrate a shell with VxWorks SSH using a socket connection between SSH and the shell.

The following code examples describe standard-input and standard-output communication in the shell process. Standard-input data sent from the client is forwarded to the shell over the TCP connection. In the same manner, data which is output from the shell on standard-output must be sent to the SSH server over the TCP connection. If the SSH client disconnects, the SSH module closes the TCP connection. Conversely, if the shell closes the TCP connection, SSH disconnects the SSH client.

This model is useful if you have an existing telnet server and want to use an SSH connection instead of a telnet connection. However, if you are using this model, a routine that establishes a TCP connection to the telnet server must be implemented and this routine must be registered by calling **ipcom_set_start_shell_ptr( )**. Thereafter, you can connect to the system using SSH and use the shell through the TCP connection over loopback to the telnet server.

The telnet server typically needs to be modified to listen on localhost only. For security reasons, it is important that this connection is only established over the loopback interface (localhost), and any authentication must be turned off in the telnet server as SSH handles the authentication.

In the following example, the **ssh_socket_shell_example** function must be registered by calling **ipcom_set_start_shell_ptr( )**.

```
#include <vxWorks.h>
#include <taskLib.h>
#include <sockLib.h>
#include <netinet/in.h>
#include <string.h>
#include <ipcom_shell.h>
#include <ipcom_err.h>
int echo_shell(int sock)
{
   char buf[256];
   int len;
   char msg[] = "Welcome to echo shell!\n";
   write(sock, msg, strlen(msg));
   while(1)
   {
      len = read(sock, buf, sizeof(buf));
      if (len <= 0)
```

```
         return OK;
      if (write(sock, buf, len) != len)
         return ERROR;
   }
   return OK;
}

int
ssh_socket_shell_example(int *stdio_sock, Ipcom_shell_ctx* sh_ctx)
{
  int ssh_sock = -1;
  int listen_sock = -1;
  int sh_sock = -1;
  int rc = ERROR;
  short port;
  socklen_t len;
  struct sockaddr_in sa;
  listen_sock = socket(AF_INET, SOCK_STREAM, 0);
  if (listen_sock == -1)
    {
     return IPCOM_ERR_FAILED;
    }
  memset(&sa, 0, sizeof(sa));
  sa.sin_len = sizeof(struct sockaddr_in);
  sa.sin_family = AF_INET;
  sa.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
  sa.sin_port = 0; /* Get an empheral port */
  if (bind(listen_sock, (struct sockaddr *)&sa, sizeof(sa)) != OK)
    {
     goto cleanup;
    }
  if (listen(listen_sock, 1) != OK)
    {
     goto cleanup;
    }
  /* Fetch the empheral port assigned to our socket */
  len = sizeof(sa);
  if (getsockname(listen_sock, (struct sockaddr*) &sa, &len) != OK)
    {
     goto cleanup;
    }
  port = sa.sin_port;
  ssh_sock = socket(AF_INET, SOCK_STREAM, 0);
  if (ssh_sock == -1)
    {
     goto cleanup;
    }
  memset(&sa, 0, sizeof(sa));
  sa.sin_len = sizeof(struct sockaddr_in);
  sa.sin_family = AF_INET;
  sa.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
  sa.sin_port = port; /* Already in network order */
  if (connect(ssh_sock, (struct sockaddr *)&sa, sizeof(sa)) != OK)
    {
     goto cleanup;
    }
  sh_sock = accept(listen_sock, NULL, NULL);
```

```
  if (sh_sock == -1)
    {
     goto cleanup;
    }
  *stdio_sock = ssh_sock;
  taskSpawn("echo_shell", 200, 0, 4*1024, echo_shell, sh_sock, 2, 3, 4, 5, 6, 7, 8, 9, 10);
  rc = IPCOM_SUCCESS;
cleanup:
     if (listen_sock != -1)
        close(listen_sock);
     if (rc != IPCOM_SUCCESS)
     {
        if (ssh_sock != -1)
           close(ssh_sock);
        if (sh_sock != -1)
           close(sh_sock);
     }
     return rc;
}
```

Using this model, you can easily convert an existing telnet server to an SSH server. For more information, see the API reference entries.

Parent topic: Integration with Shells on page 23

6.3. Multiple Shells and Ports

Wind River SSH supports multiple shells or the ability to listen to multiple ports.

You can configure the ports that a system listens to using system variables. For more information, see Run-time Configuration on page 23.

To start different shells based on the port the client uses to connect, the shell start routine registered by calling **ipcom_set_start_shell_ptr( )** must look at what port the client connected to and launch the appropriate shell. To get the peer address and port of the SSH client, call **ipcom_shell_ctx_get_peer_addr( )**.

Parent topic: Integration with Shells on page 23

7. Server Side SSH Subsystem

VxWorks SSH maintains a list of SSH subsystem names and init function pointers.

When implementing the server side of an SSH subsystem, an initialization routine must be registered for the subsystem name. This is done using **ipssh_register_subsystem( )**. When a subsystem request is received from the SSH client, the SSH server invokes the registered initialization routine for the requested subsystem. The init routine must initialize the subsystem and establish a TCP connection to the subsystem. The SSH server and the subsystem then convey data over the TCP connection.

8. Client Side SSH Subsystem

To establish an SSH subsystem client connection, you must use specific functions.

ipssh_subsystem_clt_open( )

Establishes a client connection with the server.

ipssh_subsystem_clt_read( )

Reads data from the connection.

ipssh_subsystem_clt_write( )

Writes data to the connection.

9. Methods to Start and Stop SSH

VxWorks SSH is automatically started at boot time.

You can start and stop the SSH server module using the **ipd** framework of IPCOM. The framework includes a shell command, **ipd**, as well as an API defined in **ipcom_ipd.h**.

10. Creating an SFTP Client Connection

VxWorks SSH includes an SFTP client for connecting to an external SFTP server.

About This Task

The API for the client includes routines for standard file transfer operations such as connecting to a server and manipulating files and directories.

All the routines use an **Ipssh_conn_st** structure to manage the connection to the server.

To create a new connection, perform the following steps:

Procedure

1. Call **ipssh_sftp_clt_set_userid( )** to set the user name for the connection.
2. Call **ipssh_sftp_clt_set_pw( )** or **ipssh_sftp_clt_set_key( )** to set either the password or public key.

   Note: The **ipssh_sftp_clt_set_key( )** function supports both DSA and RSA keys.

3. Call **ipssh_sftp_clt_open( )** to open the connection.

   The other **ipssh_sftp_clt_*( )** functions use the open connection. See the API reference entries for details.

Next Steps

Note: When you want to obtain a file from an SFTP server, you must explicitly supply a full path name in the **ipssh_sftp_clt_get( )** function, or set the remote path explicitly though a call to the **ipssh_sftp_clt_chdir( )** function.

11. SFTP Read Request Configuration

The SFTP client can execute multiple, non-overlapping read requests to the SFTP server.

This allows for parallel execution of read requests which reduces waiting time for both the server and the client. By default, the SFTP client uses four concurrent requests. However, the number of concurrent read requests can be configured using the ipssh.sftp.clt_max_reqs sysvar. Valid values for this sysvar are from 1 to 10.

12. File Systems and SFTP

You may wish to store or retrieve files from sources other than a traditional file system. For example, when you download updated software to a system, it is more efficient to download the new software version directly to memory without temporarily storing it in a local file system.

As another example, when large quantities of logging data are kept in memory, this must be uploaded to another system. In this case, it is convenient to retrieve the data directly from RAM without temporarily storing it as a file in the local file system.

VxWorks SSH uses a file system API to support the use of other file sources and destinations. If your application needs to download or retrieve files from another source, other than a traditional file system, you can easily implement the API provided for this purpose.

VxWorks SSH is distributed with one example implementation of the SFTP file API. This implementation uses a traditional file system as the source and destination of file operations. For more information, see **ipssh_sftp_file.c** in the *installDir* **/vxworks-7/pkgs_v2/net/ipnet-***version* **/ssh-***version* **/src** directory.

Use **ipssh_sftp_file.c** as a reference when implementing this API for other file sources.

VxWorks SSH calls **ipssh_get_sftp_func_cb( )** for each new file request. This function returns a set of function pointers to the file system that handles file operations for the path identified in the request. **ipssh_get_sftp_func_cb( )** is defined in **ipssh_config.c**. The default implementation returns pointers to routines from **ipssh_sftp_file.c**, but it can be modified to return different sets of function pointers for different directories.

Access Control on page 28 Use the callback routine **ipssh_sftp_check_access_cb( )**, defined in **ipssh_config.c**, to implement access control for SFTP operations.

12.1. Access Control

Use the callback routine **ipssh_sftp_check_access_cb( )**, defined in **ipssh_config.c**, to implement access control for SFTP operations.

Modify the function to restrict access to files and directories based on user name.

Parent topic: File Systems and SFTP on page 28

WNDRVR

5. SHELL COMMANDS

1. Command-Line Usage

VxWorks SSH includes an SSH utility and SFTP client shell commands.

SSH Utility

The SSH utility provides VxWorks shell commands to list SSH clients, stop SSH processes, and display fingerprints for RSA and DSA keys. To get help for a command, use *commandName* -h.

The commands are as follows:

- ipssh_list: List all logged in clients.
- ipssh_stop: Stop a particular spawned SSH server process or all spawned processes.
- keyfp: Generate a key fingerprint of an RSA or a DSA public key. The command options are as follows:
  - -f *fileName* : The file holding the key to generate a fingerprint. The input file must contain a public/private key pair.
  - -h: Generate a hex fingerprint.
  - -b: Generate a bubble-babble fingerprint.

SFTP Client

The sftp command creates a connection to an SFTP server, and starts a subsystem to process other SFTP client commands (sub-commands). To get help for the sftp command, use *commandName* -h. The sftp command options are as follows:

- -i *publicKeyFile* : File holding a key pair to be used during authentication. If a key file is not specified the user is prompted for a password.
- -f *hostBasedKeyFile* : File holding the host-based key for host-based authentication.
- -p *port* : Port to which the SFTP server listens.
- -w *password* : The password to use during authentication.
- *host* : IP address or hostname of the SFTP server.

Within an sftp session, use the help command to get help on sub-commands.